ASSOCIAZIONE GIGUIYA · TORINO

Documentazione tecnica e operativa del sito

Architettura, codice, configurazione di GitHub, Render, MongoDB Atlas e DNS Aruba, sviluppo locale su Windows, manutenzione e risoluzione dei problemi.

giguiya.it · Versione 1.1 – 27 ottobre 2026
Documento interno – non contiene password

Indice

1 Panoramica del progetto

2 Architettura e servizi online

3 Tecnologie e versioni

4 Struttura del repository

5 Frontend (Angular)

6 Backend (FastAPI)

7 Database (MongoDB)

8 Variabili d'ambiente

9 Configurazione da zero – GitHub

10 Configurazione da zero – MongoDB Atlas

11 Configurazione da zero – Render

12 Configurazione da zero – Dominio e DNS Aruba

13 Email (Resend)

14 Ambiente di sviluppo locale su Windows

15 Come fare modifiche e pubblicarle

16 Gestione quotidiana (area admin)

17 Backup e ripristino dei dati

18 Sicurezza e credenziali

19 Costi

20 Risoluzione dei problemi

21 Riferimenti rapidi

1. Panoramica del progetto

Il sito presenta l'Associazione Giguiya e gestisce soci, eventi e progetti.

Funzionalità pubbliche

- **Home**: presentazione dell'associazione, prossimi eventi, progetti, invito a diventare socio.
- **Eventi**: elenco e dettaglio degli eventi, prenotazione con email di conferma.
- **Progetti**: squadra sportiva, laboratori di comunità (tessitura, cucina), progetto Baliba con UNHCR e altri.
- **Chi siamo**: storia, finalità e statuto dell'associazione (PDF scaricabile).
- **5x1000**: come destinare il 5x1000 all'associazione.
- **Diventa socio**: domanda di iscrizione, con email di conferma e creazione della password.

Area socio (dopo il login)

- Dati personali, numero tessera, stato dell'iscrizione e della quota annuale, storico pagamenti, prenotazioni.

Area amministrazione (solo staff)

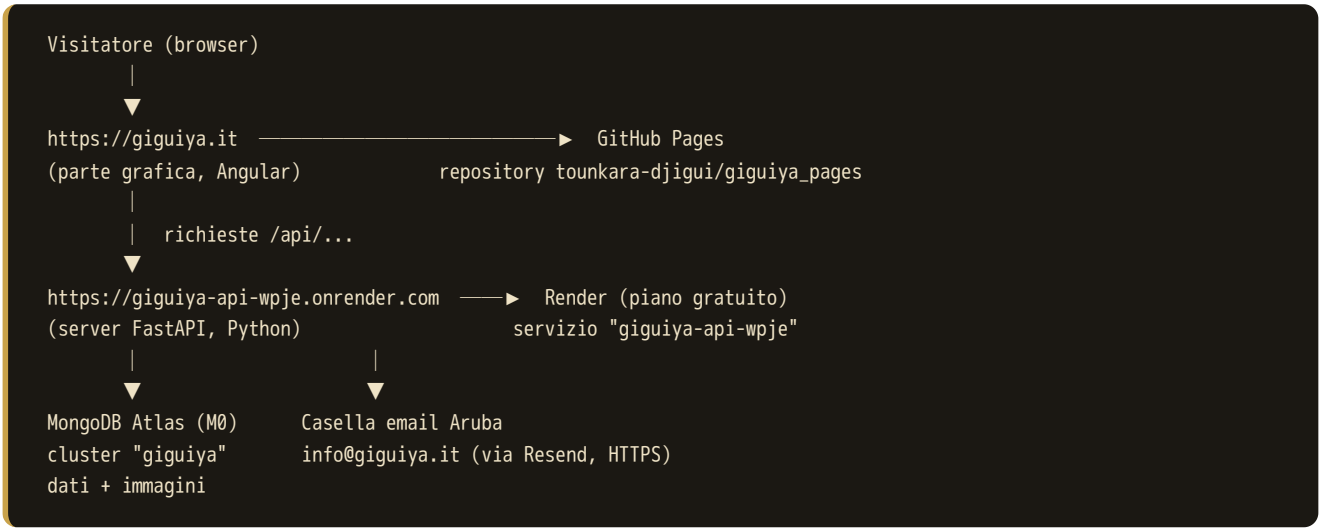
- Dashboard con statistiche.
- **Soci**: elenco, dettaglio, approvazione, registrazione dei pagamenti (iscrizione e quota annuale per anno), reinvio dell'invito, eliminazione.
- **Interessati**: persone che hanno prenotato eventi senza essere socie.
- **Eventi**: creazione, modifica, eliminazione, caricamento immagini, elenco prenotazioni.
- **Progetti**: creazione, modifica, eliminazione, caricamento immagini.

Ruoli

Ruolo	Chi è	Cosa può fare
Visitatore	chiunque	vedere il sito, prenotare eventi, chiedere l'iscrizione
Socio (`member`)	iscritto con password	area socio, prenotazioni
Admin (`admin`)	staff con email @**giguiya.it**	tutta l'area amministrazione

Gli importi di iscrizione e quota annuale sono mostrati come **"xx €"** in attesa delle cifre definitive (vedi capitolo 15.3).

2. Architettura e servizi online



Componente	Servizio	Indirizzo
Sito pubblico	GitHub Pages	https://giguiya.it
Redirect www	GitHub Pages	https://www.giguiya.it → https://giguiya.it
Vecchio indirizzo	GitHub Pages	https://tounkara-djigui.github.io/giguiya_pages → https://giguiya.it
Server (API)	Render	https://giguiya-api-wpje.onrender.com
Controllo server	Render	https://giguiya-api-wpje.onrender.com/api/health
Database	MongoDB Atlas	cluster `giguiya.7j8w7bv.mongodb.net`
Codice sorgente	GitHub	https://github.com/tounkara-djigui/giguiya_pages
Dominio e posta	Aruba	giguiya.it, info@giguiya.it

Flusso di pubblicazione

- Un invio (push) sul ramo `main` di GitHub fa partire automaticamente:
  - il workflow **"Pubblica su GitHub Pages"**, che ricompila e pubblica il sito;
  - il **deploy automatico su Render**, che aggiorna il server.

3. Tecnologie e versioni

Area	Tecnologia	Versione
Frontend	Angular (componenti standalone)	20.3
Frontend	Angular CLI	20.3
Frontend	TypeScript	5.9
Frontend	RxJS	7.8
Frontend	Stili	SCSS (tema scuro con accenti oro)
Strumenti	Node.js	20 LTS
Strumenti	Yarn	1.22
Backend	Python	3.11
Backend	FastAPI / Starlette	0.110.1 / 0.37.2
Backend	Uvicorn	0.25.0
Backend	Motor (MongoDB asincrono) / PyMongo	3.7.1 / 4.18.2
Backend	Pydantic	2.14
Backend	bcrypt (password) / PyJWT (token)	4.1.3 / 2.15.1
Database	MongoDB Atlas	piano M0 gratuito
Hosting	GitHub Pages + GitHub Actions	–
Hosting	Render (Web Service Python)	piano Free

Le versioni esatte delle librerie Python per Render sono in `backend/requirements-render.txt`. Quelle del frontend sono in `frontend/package.json` e `frontend/yarn.lock`.

4. Struttura del repository

```
giguiya_pages/
├── .github/workflows/deploy.yml    Pubblicazione automatica su GitHub Pages
├── render.yaml                     Configurazione del server su Render (Blueprint)
├── RENDER.md                       Guida breve Render/Atlas
├── docs/                           Questa documentazione (MD e PDF)
├── backend/                        Server FastAPI
│   ├── server.py                    Avvio app, CORS, registrazione delle rotte /api
│   ├── requirements-render.txt      Librerie Python per Render e per uso locale
│   ├── requirements.txt             Librerie dell'ambiente di sviluppo Emergent
│   ├── app/
│   │   ├── core/                    config.py, database.py, security.py
│   │   ├── routers/                 auth, events, members, projects, files, admin/
│   │   ├── schemas/                 modelli dei dati (Pydantic)
│   │   ├── services/                email, modelli email, eventi, soci, file, token
│   │   └── seed.py                  admin iniziale e contenuti di esempio
│   └── tests/                       test automatici
└── frontend/                       Sito Angular
    ├── angular.json, package.json
    ├── scripts/set-env.js          Scrive l'indirizzo del server in environment.ts
    ├── public/                     immagini, favicon, statuto PDF
    └── src/
        ├── index.html, main.ts, styles.scss
        ├── styles/                 _tokens.scss (colori, font), _base.scss, _admin.scss
        └── app/
            ├── app.routes.ts       elenco delle pagine
            ├── core/               config, contenuti, guard, interceptor, modelli, servizi
            ├── features/           pagine (home, eventi, progetti, chi-siamo, admin, ...)
            └── shared/             navbar, footer, logo, card evento, pipe, ...
```

5. Frontend (Angular)

5.1 Pagine (rotte)

Indirizzo	Pagina	Accesso
`/`	Home	pubblico
`/eventi`, `/eventi/:slug`	Elenco e dettaglio eventi	pubblico
`/progetti`, `/progetti/:slug`	Elenco e dettaglio progetti	pubblico
`/chi-siamo`	Chi siamo e statuto	pubblico
`/5x1000`	5x1000	pubblico
`/diventa-socio`	Domanda di iscrizione	pubblico
`/accesso`	Login	pubblico
`/accesso/registrazione-admin`	Registrazione staff (@giguiya.it)	pubblico
`/accesso/verifica-email`	Verifica email	link da email
`/accesso/imposta-password`	Creazione password	link da email
`/accesso/password-dimenticata`	Recupero password	pubblico
`/area-socio`	Area socio	socio
`/admin`, `/admin/soci`, `/admin/interessati`, `/admin/eventi`, `/admin/progetti`	Amministrazione	admin

Le rotte sono definite in `src/app/app.routes.ts`. Le pagine protette usano `memberGuard` e `adminGuard` (`core/guards/role.guard.ts`).

5.2 File da conoscere

File	Contenuto
`core/config.ts`	nome, email, telefono, indirizzo, importi (`enrollmentFee`, `annualFee`), dominio staff
`core/content/association.content.ts`	testi di Chi siamo, statuto, immagini, valori
`core/services/*.service.ts`	chiamate al server (auth, soci, admin, contenuti pubblici)
`core/interceptors/auth.interceptor.ts`	invio dei cookie e rinnovo automatico della sessione
`shared/components/navbar`, `footer`	menu in alto e fondo pagina
`styles/_tokens.scss`	colori, font, spaziature del tema
`public/images/`, `public/docs/`	immagini e statuto PDF

5.3 Indirizzo del server

- `scripts/set-env.js` legge `REACT_APP_BACKEND_URL` (dal file `frontend/.env` o dall'ambiente) e scrive `src/environments/environment.ts`.
- Il comando parte da solo con `yarn start` e `yarn build`.
- Su GitHub Pages il valore viene dal workflow: la variabile `API_URL` del repository se esiste, altrimenti `https://giguiya-api-wpje.onrender.com`.

5.4 Pubblicazione su GitHub Pages (`.github/workflows/deploy.yml`)

1. Installa le dipendenze con Yarn (Node 20).
2. Compila con `ng build --configuration production --base-href "/"`.
3. Copia `index.html` in `404.html`, così i link diretti (es. `/eventi`) funzionano.
4. Pubblica la cartella `dist/.../browser` su GitHub Pages.

I percorsi di immagini e documenti sono relativi (`images/...`, `docs/...`), quindi il sito funziona sia sul dominio sia in un sottopercorso.

6. Backend (FastAPI)

6.1 Endpoint principali (tutti con prefisso `/api`)

Metodo	Percorso	Uso	Accesso
GET	`/health`	controllo stato	pubblico
POST	`/auth/login`, `/auth/logout`, `/auth/refresh`	sessione	pubblico
GET	`/auth/me`	utente collegato	autenticato
POST	`/auth/admin/register`	registrazione staff @giguiya.it	pubblico
POST	`/auth/verify-email`, `/auth/set-password`, `/auth/forgot-password`	link email	pubblico
POST	`/auth/change-password`	cambio password	autenticato
GET	`/events`, `/events/{slug}`	eventi	pubblico
POST	`/events/{slug}/bookings`	prenotazione	pubblico
GET	`/projects`, `/projects/{slug}`	progetti	pubblico
POST	`/members/register`	domanda di iscrizione	pubblico
GET	`/members/me`	area socio	socio
GET	`/files/{path}`	immagini caricate	pubblico
POST	`/admin/uploads`	caricamento immagini (max 8 MB)	admin
GET	`/admin/stats`	statistiche	admin
GET/PATCH/DELETE	`/admin/members/...`	gestione soci	admin
PUT	`/admin/members/{id}/fees/{anno}`	pagamento quota	admin
POST	`/admin/members/{id}/resend-invite`	reinvio invito	admin
GET	`/admin/contacts`	interessati	admin
GET/POST/PUT/DELETE	`/admin/events/...`, `/admin/bookings/{id}`	eventi e prenotazioni	admin
GET/POST/PUT/DELETE	`/admin/projects/...`	progetti	admin

La documentazione interattiva delle API si trova su `https://giguiya-api-wpje.onrender.com/docs`.

6.2 Sicurezza

- **Password** salvate cifrate con bcrypt.

- **Sessione** con token JWT in **cookie httpOnly** (non leggibili da JavaScript):
  - accesso valido 60 minuti, rinnovo automatico per 7 giorni;
  - cookie `Secure` e `SameSite=None`.
- **Blocco tentativi**: dopo 5 login errati l'accesso è bloccato per 15 minuti.
- **Staff**: ci si registra solo con email @giguiya.it, confermata via email. Dopo la conferma l'account resta **in attesa** finché l'amministratore principale (`ADMIN_EMAIL`) non lo approva dalla pagina **Admin → Staff**. Solo l'amministratore principale può approvare, revocare o eliminare gli account staff.
- **CORS**: il server accetta richieste solo dal sito indicato in `FRONTEND_URL`, cioè https://giguiya.it, e da `http://localhost:3000` per lo sviluppo.
- **Email**: i testi inviati vengono controllati e ripuliti prima dell'invio.

6.3 Avvio del server (`seed.py`)

A ogni avvio il server:

1. crea gli indici del database;
2. crea l'amministratore `ADMIN_EMAIL` se non esiste;
   - se la password salvata è diversa da `ADMIN_PASSWORD`, la riallinea;
3. se progetti o eventi sono vuoti, inserisce i contenuti di esempio.

> Importante: la password dell'amministratore principale si cambia modificando **ADMIN_PASSWORD su Render**. Una password cambiata dal sito verrebbe sovrascritta al riavvio successivo.

6.4 Email e file

- **Email**: se `RESEND_API_KEY` è impostata il server invia tramite l'API di Resend (HTTPS). In alternativa usa SMTP (`SMTP_HOST`), che però **non funziona sul piano gratuito di Render**, perché le porte SMTP sono bloccate. Senza nessuna delle due usa il servizio email dell'ambiente di sviluppo Emergent.
- **Immagini caricate**: senza `EMERGENT_LLM_KEY` vengono salvate nel database MongoDB (GridFS, raccolta `uploads`). È il caso di Render.

7. Database (MongoDB)

Nome database: `giguiya`.

Raccolta	Contenuto
`users`	utenti (admin e soci): email, password cifrata, ruolo, dati anagrafici, tessera, quote
`events`	eventi: titolo, slug, date, luogo, descrizione, immagine, posti
`bookings`	prenotazioni agli eventi (una per email ed evento)
`projects`	progetti: titolo, slug, descrizione, immagine, categoria
`files`	elenco delle immagini caricate
`uploads.files`, `uploads.chunks`	contenuto delle immagini (GridFS)
`auth_tokens`	token dei link email (verifica, password)
`login_attempts`	conteggio dei tentativi di login errati

8. Variabili d'ambiente

8.1 Backend (su Render: servizio `giguiya-api-wpje` → Environment)

Variabile	Valore	Note
`PYTHON_VERSION`	`3.11.9`	versione di Python su Render
`MONGO_URL`	`mongodb+srv://tounkaradjg_db_user:PASSWORD@giguiya.7j8w7bv.mongodb.net/?compressors=zlib`	**segreto**
`DB_NAME`	`giguiya`	
`JWT_SECRET`	generato da Render	**segreto**; se cambia, tutti devono rifare il login
`FRONTEND_URL`	`https://giguiya.it`	link nelle email e CORS
`ADMIN_EMAIL`	`info@giguiya.it`	amministratore principale
`ADMIN_PASSWORD`	password del sito	**segreto**; diversa da quella della posta
`ADMIN_EMAIL_DOMAIN`	`giguiya.it`	dominio obbligatorio per lo staff
`EMAIL_FROM_NAME`	`Giguiya`	nome del mittente
`EMAIL_REPLY_TO`	`info@giguiya.it`	indirizzo di risposta
`RESEND_API_KEY`	chiave API di Resend (`re_...`)	**segreto**
`EMAIL_FROM`	`info@giguiya.it`	mittente, sul dominio verificato in Resend

8.2 Frontend

Dove	Variabile	Valore
GitHub → Settings → Secrets and variables → Actions → Variables (facoltativa)	`API_URL`	indirizzo del server senza `/` finale
`frontend/.env` (solo in locale)	`REACT_APP_BACKEND_URL`	`http://localhost:8001`

I file `.env` non vengono mai inviati a GitHub (sono esclusi da `.gitignore`).

9. Configurazione da zero – GitHub

1. Create l'account GitHub e il repository `giguiya_pages`, con ramo principale `main`.
2. Inviate il codice:
   - da Emergent con **Save to GitHub**;
   - oppure da un computer con `git push` (vedi capitolo 14).
3. **Settings** → **Pages** → **Build and deployment** → **Source**: scegliete **GitHub Actions**.
4. **Settings** → **Pages** → **Custom domain**: scrivete `giguiya.it` e premete **Save**. Aspettate **DNS check successful**.
5. Spuntate **Enforce HTTPS**: il certificato si attiva da solo, di solito entro un'ora.
6. **Actions** → **"Pubblica su GitHub Pages"**: ogni push su `main` pubblica il sito. Si può avviare anche a mano con **Run workflow**.
7. Facoltativo: **Settings** → **Secrets and variables** → **Actions** → **Variables** → `API_URL`, solo se l'indirizzo del server cambia.

10. Configurazione da zero – MongoDB Atlas

1. Create un account su https://www.mongodb.com/cloud/atlas. Va bene l'accesso con GitHub.
2. Create un progetto (`Giguiya`) e un cluster **M0 Free** in una regione europea, con nome `giguiya`.
3. **Database Access** → **Add New Database User**:
   - nome utente: `tounkaradjg_db_user`;
   - password: **solo lettere e numeri**, perché i simboli `@ : / ? # %` rompono la stringa;
   - ruolo: **Read and write to any database**.
4. **Network Access** → **Add IP Address**: `0.0.0.0/0`. Serve perché Render non ha un indirizzo fisso.
5. **Connect** → **Drivers** → **Python**: copiate la stringa e sostituite la password:

```
mongodb+srv://tounkaradjg_db_user:PASSWORD@giguiya.7j8w7bv.mongodb.net/?compressors=zlib
```

La parte dopo i due punti è la password. La `@` separa la password dall'indirizzo del cluster: non va tolta.

6. Il database `giguiya` e le raccolte vengono creati dal server al primo avvio.

11. Configurazione da zero – Render

11.1 Con Blueprint (consigliato)

1. Create un account su https://render.com con accesso tramite GitHub.
2. **+ New** → **Blueprint** → repository `giguiya_pages`, ramo `main`.
3. Render legge `render.yaml` e propone il servizio web. Compilate i valori segreti del capitolo 8.1.
4. **Apply**. Dopo 3–5 minuti lo stato diventa **Live**.
5. Verifica: `https://<servizio>.onrender.com/api/health` → `{"status":"ok"}`.

11.2 Manuale (alternativa)

Campo	Valore
Tipo	Web Service, runtime Python, piano Free
Root Directory	`backend`
Build Command	`pip install -r requirements-render.txt`
Start Command	`uvicorn server:app --host 0.0.0.0 --port $PORT`
Health Check Path	`/api/health`
Environment	tutte le variabili del capitolo 8.1

11.3 Uso quotidiano

- **Logs**: messaggi ed errori del server.
- **Events**: storico dei deploy.
- **Manual Deploy** → **Deploy latest commit**: riavvia con l'ultimo codice.
- **Environment**: modifica delle variabili. Dopo **Save Changes** Render riavvia il server.
- Sul piano Free il server si spegne dopo circa 15 minuti senza visite. La prima richiesta successiva impiega fino a un minuto.

12. Configurazione da zero – Dominio e DNS Aruba

Pannello: **Aruba** → **Area Clienti** → **Domini** → **giguiya.it** → **Gestione DNS e Name Server**. Name server in uso: Aruba (`dns.technorail.com`, `dns2.technorail.com`, `dns3.arubadns.net`, `dns4.arubadns.cz`).

12.1 Record per il sito (GitHub Pages)

Tipo	Nome host	Valore	TTL
A	`@`	`185.199.108.153`	1 ora
A	`@`	`185.199.109.153`	1 ora
A	`@`	`185.199.110.153`	1 ora
A	`@`	`185.199.111.153`	1 ora
CNAME	`www`	`tounkara-djigui.github.io`	1 ora

Facoltativi, per IPv6: AAAA `@` → `2606:50c0:8000::153`, `2606:50c0:8001::153`, `2606:50c0:8002::153`, `2606:50c0:8003::153`.

Valori precedenti, sostituiti: A `@` e A `www` → `31.11.35.15` (hosting Aruba).

12.2 Record da NON modificare (posta e servizi Aruba)

Tipo	Nome host	Valore
MX	`@`	`mx.giguiya.it` (priorità 10)
A	`mx`, `mail`, `smtp`, `pop`, `imap` e simili	`62.149.128.x` (server di posta Aruba)
TXT	`@`	`"v=spf1 include:_spf.aruba.it ~all"`
TXT	`_dmarc`	`"v=DMARC1; p=none; adkim=r; aspf=r;"`
SRV	`_autodiscover._tcp`	`0 0 443 autodiscover.aruba.it`
SRV	`_xmpp-client._tcp`, `_xmpp-server._tcp`	`imchat1.aruba.it`
CNAME	`stat`, `statistiche`	`redirectstat.aruba.it`
A	`localhost`	`127.0.0.1`

12.3 Verifica

- https://giguiya.it si apre con il lucchetto.
- https://www.giguiya.it porta a https://giguiya.it.
- Le modifiche DNS si propagano in un tempo da qualche minuto a qualche ora.
- Controllo dall'esterno: https://dns.google/query?name=giguiya.it

12.4 Facoltativo: `api.giguiya.it`

Per far funzionare il login anche su iPhone/Safari, il server può avere un indirizzo sullo stesso dominio:

1. Render → servizio → **Settings** → **Custom Domains** → **Add** `api.giguiya.it`.

2. Aruba: aggiungete un record CNAME `api` → `giguiya-api-wpje.onrender.com`.
3. GitHub: impostate la variabile `API_URL` = `https://api.giguiya.it` e rilanciate il workflow.

13. Email (Resend)

Il piano gratuito di Render blocca le porte SMTP (25, 465, 587), quindi il sito non può inviare dalla casella Aruba. Le email partono tramite **Resend** (gratuito fino a 3.000 email al mese), con mittente `info@giguiya.it`. Le risposte dei soci arrivano comunque nella casella Aruba, grazie a `EMAIL_REPLY_TO`.

Configurazione

1. Create un account su https://resend.com.
2. **Domains** → **Add Domain** → `giguiya.it`, regione Europa.
3. Resend mostra alcuni record DNS: TXT per DKIM (`resend._domainkey`), MX e TXT per `send`. Inseriteli **esattamente** nel pannello DNS di Aruba.
   - Non create un secondo record SPF su `@` e non toccate il record MX principale della posta.
4. Premete **Verify** in Resend e aspettate lo stato **Verified**.
5. **API Keys** → **Create API Key** con permesso *Sending access*. Copiatela: viene mostrata una sola volta.
6. Su Render, in **Environment**, impostate `RESEND_API_KEY` e `EMAIL_FROM=info@giguiya.it`, poi salvate.

Email inviate dal sito

- conferma della domanda di iscrizione e link per creare la password;
- verifica dell'email dello staff;
- avviso all'amministratore principale per ogni nuova richiesta staff confermata;
- avviso allo staff quando viene approvato;
- recupero password;
- conferma di prenotazione degli eventi.

Dove controllare: Resend → **Emails**, per lo stato di ogni invio; Render → **Logs**, cercando `Resend send failed`.

Se le email finiscono nello spam, verificate su Aruba che il record SPF sia presente e, se disponibile nel pannello, attivate la firma **DKIM** della casella.

14. Ambiente di sviluppo locale su Windows

14.1 Programmi da installare

Programma	Dove scaricarlo	Note
Git	https://git-scm.com/download/win	opzioni predefinite
Node.js 20 LTS	https://nodejs.org	include npm
Yarn 1	da terminale: `npm install -g yarn`	
Python 3.11	https://www.python.org/downloads/windows	spuntate **Add python.exe to PATH**
Visual Studio Code	https://code.visualstudio.com	estensioni consigliate: Angular Language Service, Python
MongoDB Compass (facoltativo)	https://www.mongodb.com/products/compass	per vedere i dati

Verifica, in PowerShell:

```
git --version
node -v
yarn -v
python --version
```

14.2 Scaricare il progetto

```
cd $HOME\Documents
git clone https://github.com/tounkara-djigui/giguiya_pages.git
cd giguiya_pages
```

14.3 Database per lo sviluppo

Non usate il database `giguiya` di produzione per le prove. Le scelte possibili sono due:

- **A (consigliata)**: stesso cluster Atlas, ma `DB_NAME=giguiya_dev`. Per collegarvi dal vostro PC, aggiungete in Atlas, sotto Network Access, anche il vostro IP.
- **B**: installare MongoDB Community Server (https://www.mongodb.com/try/download/community) e usare `MONGO_URL=mongodb://localhost:27017`.

14.4 Backend

```
cd backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements-render.txt
```

Se PowerShell blocca l'attivazione, eseguite una volta: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`

Create il file `backend\.env`:

```
MONGO_URL=mongodb+srv://tounkaradjg_db_user:PASSWORD@giguiya.7j8w7bv.mongodb.net/?compressors=zlib
DB_NAME=giguiya_dev
JWT_SECRET=una-frase-lunga-e-casuale-di-almeno-32-caratteri
FRONTEND_URL=http://localhost:3000
ADMIN_EMAIL=info@giguiya.it
ADMIN_PASSWORD=PasswordDiProva123
ADMIN_EMAIL_DOMAIN=giguiya.it
EMAIL_FROM_NAME=Giguiya
EMAIL_REPLY_TO=info@giguiya.it
SMTP_HOST=smtps.aruba.it
SMTP_PORT=465
SMTP_USER=info@giguiya.it
SMTP_PASSWORD=PASSWORD_CASELLA
SMTP_FROM=info@giguiya.it
```

In locale, dal vostro PC, la casella Aruba via SMTP funziona. In alternativa usate `RESEND_API_KEY` ed `EMAIL_FROM`, come su Render.

Avvio:

```
uvicorn server:app --reload --port 8001
```

Controllo: http://localhost:8001/api/health. La documentazione delle API è su http://localhost:8001/docs.

14.5 Frontend

In un secondo terminale:

```
cd frontend
yarn install
```

Create il file `frontend\.env`:

```
REACT_APP_BACKEND_URL=http://localhost:8001
```

Avvio:

```
yarn start
```

Aprite http://localhost:3000. Il sito si aggiorna da solo a ogni salvataggio.

> Attenzione: `yarn start` riscrive `src/environments/environment.ts` con l'indirizzo locale. Questo file non va inviato a GitHub con l'indirizzo locale. Il workflow lo rigenera comunque in fase di pubblicazione.

14.6 Compilazione di prova

```
yarn build
```

Il risultato si trova in `frontend\dist\`.

15. Come fare modifiche e pubblicarle

15.1 Flusso di lavoro

1. Aggiornate la copia locale: `git pull`.
2. Fate le modifiche e provatele in locale (capitolo 14).
3. Salvate e inviate:

```
git add .
git commit -m "Descrizione della modifica"
git push origin main
```

4. GitHub Pages si aggiorna in 2–3 minuti (scheda **Actions**). Render si aggiorna in 3–5 minuti (scheda **Events**).

15.2 Modifiche frequenti

Cosa cambiare	File
Email, telefono, indirizzo	`frontend/src/app/core/config.ts`
Testi di Chi siamo e statuto	`frontend/src/app/core/content/association.content.ts`, `features/about/`
Testi della Home	`frontend/src/app/features/home/home.page.html`
Pagina 5x1000	`frontend/src/app/features/five-per-mille/`
Menu e fondo pagina	`frontend/src/app/shared/components/navbar`, `footer`
Colori e font	`frontend/src/styles/_tokens.scss`
Immagini fisse	`frontend/public/images/`, nello stesso formato e con lo stesso nome
Statuto PDF	`frontend/public/docs/statuto-aspcmt-giguiya.pdf`
Testi delle email	`backend/app/services/email_templates.py`
Eventi e progetti	dall'area **admin** del sito, senza modificare il codice

15.3 Impostare gli importi reali (oggi "xx €")

1. `frontend/src/app/core/config.ts`: `enrollmentFee: '5'`, `annualFee: '60'` (esempio).
2. `backend/app/core/config.py`:
   - `ENROLLMENT_FEE_LABEL = "5"` e `ANNUAL_FEE_LABEL = "60"`;
   - `ENROLLMENT_FEE` e `ANNUAL_FEE`, gli importi numerici registrati nei pagamenti.
3. Inviate le modifiche (15.1).

15.4 Aggiungere una pagina

1. Create il componente:

```
cd frontend
npx ng generate component features/nuova/nuova.page --skip-tests
```

2. Aggiungete la rotta in `src/app/app.routes.ts`:

```
{ path: 'nuova', loadComponent: () => import('./features/nuova/nuova.page'), title: 'Nuova · Giguiya' },
```

Il componente deve avere `export default`, come le altre pagine.

3. Aggiungete il link nel menu, in `shared/components/navbar`.

15.5 Aggiungere un endpoint al server

1. Create o modificate un file in `backend/app/routers/`. Il prefisso `/api` viene aggiunto in `server.py`.
2. Definite i dati in `backend/app/schemas/`.
3. Registrate il router in `server.py`, se è nuovo.
4. Se aggiungete librerie Python, inseritele anche in `requirements-render.txt` con la versione esatta.

15.6 Aggiornare Angular

Aggiornate una versione principale alla volta:

```
cd frontend
npx ng update
npx ng update @angular/core@21 @angular/cli@21
yarn build
```

Guida ufficiale: https://angular.dev/update-guide

16. Gestione quotidiana (area admin)

1. Andate su https://giguiya.it/accesso e accedete con l'account admin.
2. **Soci**:
   - aprite un socio per vederne i dati;
   - segnate l'iscrizione come pagata;
   - registrate la quota per l'anno;
   - reinviate l'invito se il socio non ha ricevuto l'email.
3. **Eventi**: **Nuovo evento** → titolo, date, luogo, descrizione, posti, immagine (caricamento dal computer, max 8 MB). Il pulsante con l'icona delle persone mostra le prenotazioni.
4. **Progetti**: stessa logica degli eventi. Oltre all'immagine principale potete caricare una **galleria** fino a 12 foto, che i visitatori sfogliano a schermo intero.
5. **Nuovi membri dello staff**:
   - si registrano su `/accesso/registrazione-admin` con un'email @giguiya.it e confermano dal link ricevuto;
   - l'amministratore principale riceve un'email di avviso;
   - in **Admin** → **Staff** l'amministratore principale preme **Approva**, oppure **Revoca** o elimina;
   - finché l'account non è approvato, lo staff non può entrare.
6. **Tema chiaro/scuro**: ogni visitatore può cambiarlo con il pulsante sole/luna nel menu. La scelta resta salvata nel suo browser.

17. Backup e ripristino dei dati

Il piano gratuito M0 di Atlas non fa backup automatici. Fate un backup almeno una volta al mese.

1. Installate **MongoDB Database Tools**: https://www.mongodb.com/try/download/database-tools
2. Backup:

```
mongodump --uri="mongodb+srv://tounkaradjg_db_user:PASSWORD@giguiya.7j8w7bv.mongodb.net/giguiya" --out=backup-AAAA-MM-GG
```

3. Ripristino, che sovrascrive i dati esistenti:

```
mongorestore --uri="mongodb+srv://tounkaradjg_db_user:PASSWORD@giguiya.7j8w7bv.mongodb.net" --drop backup-AAAA-MM-GG
```

In alternativa, con MongoDB Compass potete esportare ogni raccolta in JSON.

Conservate i backup in un luogo sicuro: contengono dati personali dei soci.

18. Sicurezza e credenziali

Credenziale	Dove si trova	Come cambiarla
Accesso GitHub	account tounkara-djigui	github.com → Settings
Accesso Atlas	account Atlas (login con GitHub)	–
Utente database	Atlas → Database Access	Edit Password, poi aggiornare `MONGO_URL` su Render
Admin del sito	Render → `ADMIN_EMAIL` / `ADMIN_PASSWORD`	cambiare su Render, che riavvia il server
Casella email	Aruba Webmail	cambiare su Aruba (non serve aggiornare Render)
Chiave Resend	Render → `RESEND_API_KEY`	creare una nuova chiave in Resend, aggiornare Render, eliminare la vecchia
Chiave delle sessioni	Render → `JWT_SECRET`	rigenerare; tutti dovranno rifare il login
Dominio e DNS	Area Clienti Aruba	–

Regole

- Le password non vanno mai scritte nel codice, su GitHub o in chat.
- Usate password diverse per sito, posta e database.
- Attivate la verifica in due passaggi su GitHub, Render, Atlas e Aruba.
- Trattate i dati dei soci nel rispetto del GDPR: niente esportazioni condivise, e i backup vanno protetti.

19. Costi

Servizio	Piano	Costo
GitHub + GitHub Pages	Free	0 €
Render	Free	0 € (server in pausa senza visite)
MongoDB Atlas	M0	0 € (512 MB)
Aruba	dominio + posta	rinnovo annuale secondo il contratto Aruba

Se in futuro serve un server sempre acceso, Render offre il piano **Starter**. Il prezzo aggiornato è sul sito di Render.

20. Risoluzione dei problemi

Sintomo	Causa probabile	Soluzione
Render: `bad auth : authentication failed`	password errata in `MONGO_URL`	nuova password solo alfanumerica in Atlas, aggiornare `MONGO_URL`
Render: `ServerSelectionTimeoutError`	Atlas blocca la connessione	Network Access: `0.0.0.0/0` attivo
Render: `KeyError: 'NOME'`	variabile mancante	aggiungerla in Environment
Il sito non mostra gli eventi	server in pausa o indirizzo API errato	attendere un minuto; verificare `/api/health`
Errore CORS nella console del browser	`FRONTEND_URL` diverso dal dominio	`FRONTEND_URL=https://giguiya.it` su Render
Login non funziona su iPhone/Safari	sito e server su domini diversi	configurare `api.giguiya.it` (12.4)
Le email non arrivano	dominio non verificato su Resend, chiave errata o spam	Resend → Emails e Domains; log di Render (`Resend send failed`); cartella spam
Lo staff non riesce ad accedere dopo la conferma	account in attesa di approvazione	l'amministratore principale lo approva in Admin → Staff
giguiya.it mostra un "404" di GitHub	dominio non impostato in Pages	Settings → Pages → Custom domain
"DNS check unsuccessful"	DNS non ancora propagati o record errati	verificare la tabella 12.1 e attendere
Password admin cambiata e poi "tornata indietro"	`ADMIN_PASSWORD` riallinea al riavvio	cambiare `ADMIN_PASSWORD` su Render
Workflow GitHub fallito	errore di compilazione	Actions → aprire l'esecuzione → leggere il passaggio rosso; provare `yarn build` in locale

21. Riferimenti rapidi

Voce	Valore
Sito	https://giguiya.it
Area admin	https://giguiya.it/admin
Server	https://giguiya-api-wpje.onrender.com
Stato server	https://giguiya-api-wpje.onrender.com/api/health
Documentazione API	https://giguiya-api-wpje.onrender.com/docs
Repository	https://github.com/tounkara-djigui/giguiya_pages
GitHub Actions	https://github.com/tounkara-djigui/giguiya_pages/actions
Render	https://dashboard.render.com
MongoDB Atlas	https://cloud.mongodb.com
Aruba	https://admin.aruba.it
IP GitHub Pages	185.199.108.153 · 185.199.109.153 · 185.199.110.153 · 185.199.111.153
Contatti associazione	info@giguiya.it · 320 155 0017 · Corso Casale 97, Torino

Nota sull'ambiente di sviluppo Emergent: il progetto è stato sviluppato su Emergent. Lì esiste ancora un'anteprima con un database separato, che usa i servizi email e file di Emergent. I dati inseriti nell'anteprima, compresi eventi e progetti modificati, **non** sono nel database Atlas di produzione.